

AER1217 Lab 4

Dejan Plavsic - 1012836442
Chris McGale - 1005303841
Mohsen Movahedi - 1005371517

1.0 Purpose

The purpose of this lab was to complete pose estimation for a vehicle traveling through a city. The pose estimation was completed via the Kabsch algorithm, in combination with the RANSAC algorithm for outlier rejection. The video being used is from the Kitti dataset [1], which features an automobile driving through a small town in Germany, with a stereo camera mounted to the roof.

2.0 Algorithms & Equations

2.1 RANSAC

The RANSAC algorithm is a popular algorithm for outlier registration. Using three random corresponding points from the 3D point cloud, from one frame relative to the next (p_j^a and p_j^b respectively), it is possible to generate a transformation matrix from this (discussed in depth in point cloud alignment). RANSAC for this implementation of generating a 3D transformation matrix requires only three data points each iteration.

The initial three data points are then assessed in quality by applying the 3D transformation matrix to all the 3D point cloud points from the initial frame, and comparing the error of the approximation against points in the next point cloud frame. The formula for calculating these estimated points is seen below, while the variables C_{ba} and r_{ba}^b discussed in Section 2.2.

$$p_j^b = C_{ba} p_j^a + r_{ba}^b$$

The error is defined by the number of inlier versus outlier points. This cycle is repeated for the same frame until some predefined threshold is met, typically based on the number of iterations, or acceptable error.

2.2 Point Cloud Alignment

Point cloud alignment is responsible for the generation of the 3D transformation matrices. The SIFT feature detection algorithm is used for feature detection (generation of points), however it is not the focus of this report, and hence is not covered in depth. The first step after acquiring the features is to use the inverse stereo camera model to convert featured points in the images to a 3D point cloud. The inverse stereo camera model can be seen below where u represents horizontal distance, v represents vertical distance, b represents distance between cameras, f represents focal length, and c represents the principal point, while subscripts l and r are used to denote left or right (camera).

$$\rho = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \mathbf{g}(\mathbf{y}) = \frac{b}{u_l - u_r} \begin{bmatrix} \frac{1}{2}(u_l + u_r) - c_u \\ \frac{f_u}{f_v}(\frac{1}{2}(v_l + v_r) - c_v) \\ f_u \end{bmatrix}$$

An algorithm commonly referred to as the Kabsch-Umeyama algorithm [2] is then used to convert the 3D points into a transformation matrix, using three points at a time. These random points are then weighted

and used to construct the W_{ba} covariance matrix seen below, where w represents weights, p_j^a / p_j^b have the same meaning as prior, and p_a / p_b are used to represent the weighted version of the points.

$$W_{ba} = \frac{1}{w} \sum_{j=1}^J w_j (p_j^b - p_b)(p_j^a - p_a)^T$$

We used a few different weighted methods including: uniform weights, inverse depth, and square root inverse depths. Both inverse depths massively improved results likely due to further away targets having more noise.

Using the covariance matrix SVD is applied to decompose the matrix into the following format

$$VSU^T = W_{ba}$$

With the decomposition of the matrix it is now hence possible to construct the rotation matrix (C_{ba}) and translation vector (r_{ba}^b) using the following formulas.

$$C_{ba} = \mathbf{V} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \det \mathbf{U} \det \mathbf{V} \end{bmatrix} \mathbf{U}^T$$

$$r_{ba}^b = -C_{ba}^T p_b + p_a$$

It is then trivial to create the full transformation matrix using the following formula.

$$\mathbf{T}_{ba} = \begin{bmatrix} C_{ba} & r_{ba}^b \\ \mathbf{0}^T & 1 \end{bmatrix}$$

3.0 Estimated Results

The results are stochastic by nature of the RANSAC algorithm randomly sampling points and arbitrarily assigning a threshold. This will mean users may not get the exact results we describe, but these were approximately consistent for most iterations tested:

The cumulative error over the motion had a mean of 1.07m, but expanded to 2m according to Figure 2. Figure 3 shows a top down view which projects out the vertical axis. This is a fair view since we are dealing with a vehicle constrained to the road. This view helps us identify an overshoot due to a positive Y bias, which does correct itself by the end of the clip. Figure 4 shows the RMSE doesn't show a significant spike as a result of a single frame but instead shows a cumulative growth towards this error from frames 25-75.

Please see *video.avi* for the video of our inliers visualized.

4.0 Discussion

4.1 VO Limitations

A Primary issue present in visual odometry (VO) using only onboard cameras is drift accumulation over time. The pose is estimated incrementally frame to frame without any global correction. Small errors in feature localization and disparity estimation compound over time, increasing the deviation from the ground truth.

VO performance is also highly dependent on feature quality and environmental conditions. In frames with low texture, repetitive patterns, or dynamic objects, feature matching becomes less reliable, increasing the outlier rates, leading to a decrease in pose estimation accuracy. This method assumes a mostly static environment and does not explicitly handle moving objects, which can introduce biased motion estimation.

Another limitation arises from the depth sensitivity in stereo systems. Depth is inversely proportional to disparity, small pixel-level disparity errors lead to large uncertainty in our reconstruction. This was partially mitigated in our implementation by thresholding disparities and weighting points by inverse depth, though inaccuracies remain.

4.2 How can it improve?

Drift can be mitigated through several onboard techniques

- Use more robust feature descriptors and an adaptive RANSAC, as well as tracking features across multiple frames rather than pairwise matching can increase estimation accuracy.
- employ temporal optimization techniques such as a sliding-window bundle adjustment, to refine poses and 3D landmarks over multiple frames, in order to reduce drift.
- Adding onboard inertial sensor measurements (visual-inertial odometry) to improve robustness, particularly in low feature environments or during rapid motion.
- Incorporate loop closure detection to allow the system to recognize previously visited locations and correct accumulated errors.

In practice, combining all these methods results in a transition from pure VO to a full SLAM framework, which provides local accuracy and globally consistency without external infrastructure.

References:

1. A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The KITTI dataset," *International Journal of Robotics Research*, vol. 32, no. 11, pp. 1231–1237, 2013.
2. "Kabsch-Umeyama Algorithm – How to Align Point Patterns," YouTube video, Dec. 2, 2023. [Online]. Available: https://www.youtube.com/watch?v=nCs_e6fP7Jo

Appendix:

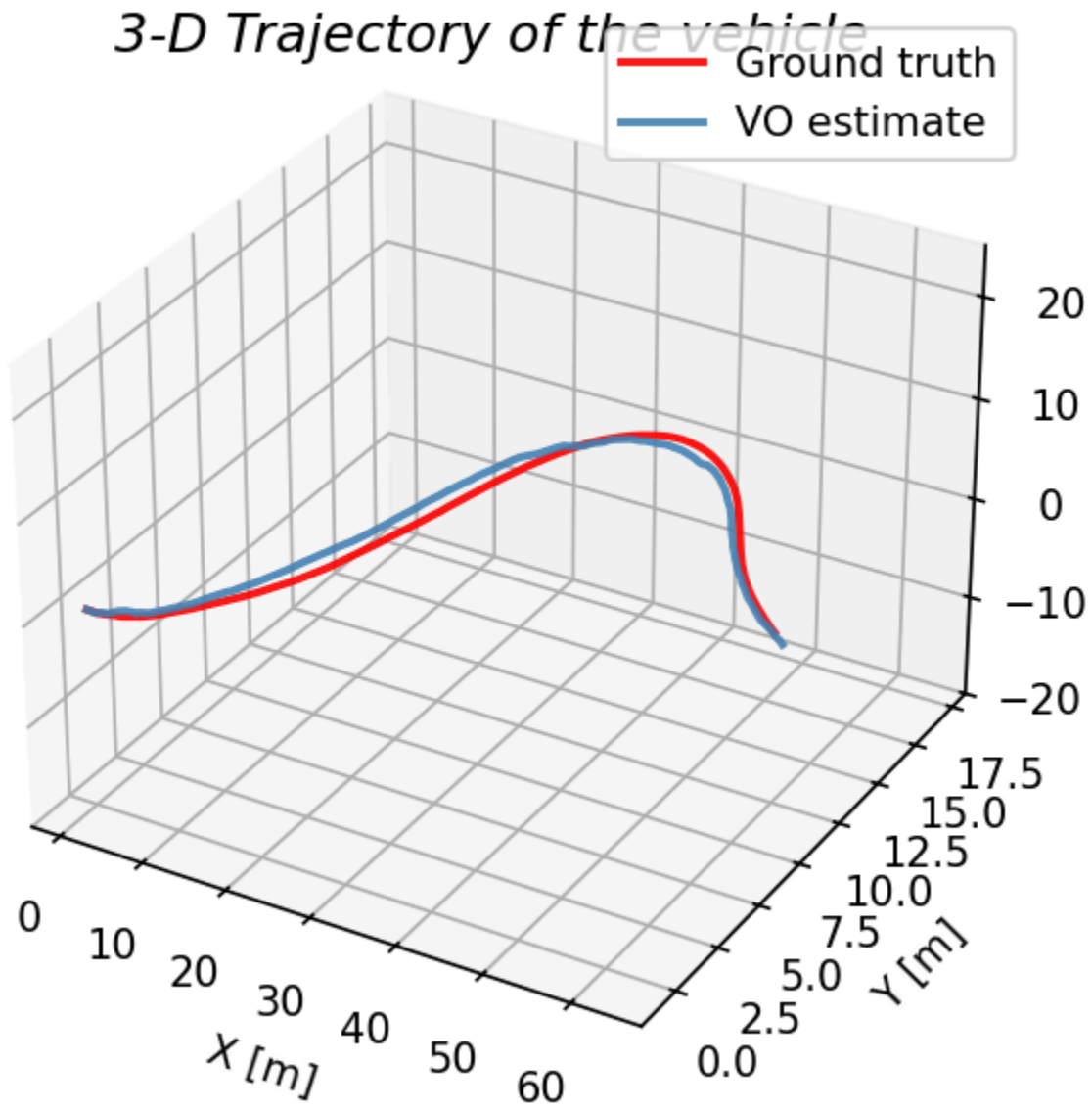


Figure 1: VO estimate of 3D trajectory vs ground truth

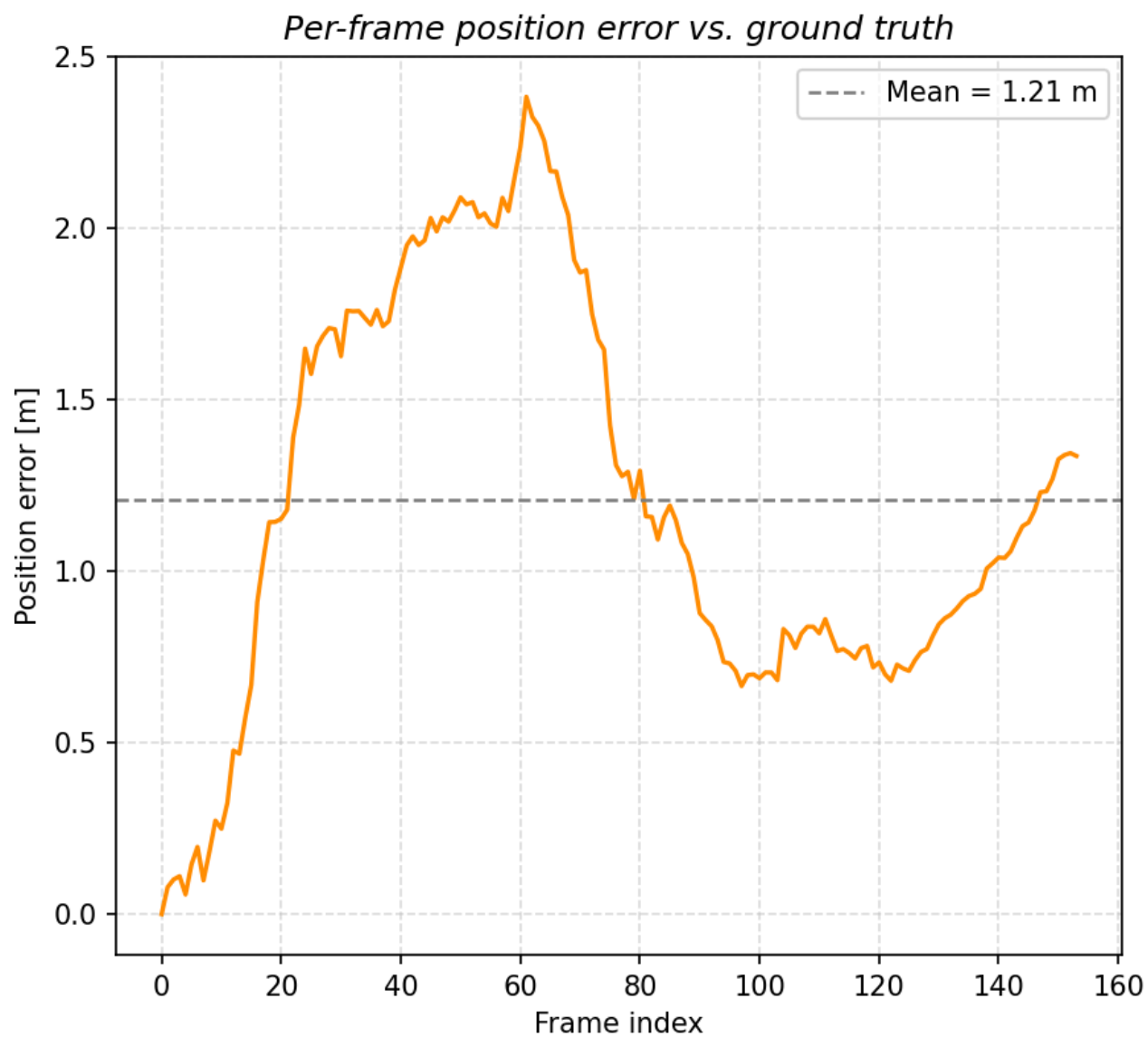


Figure 2: Difference between VO estimate and ground truth (3D)

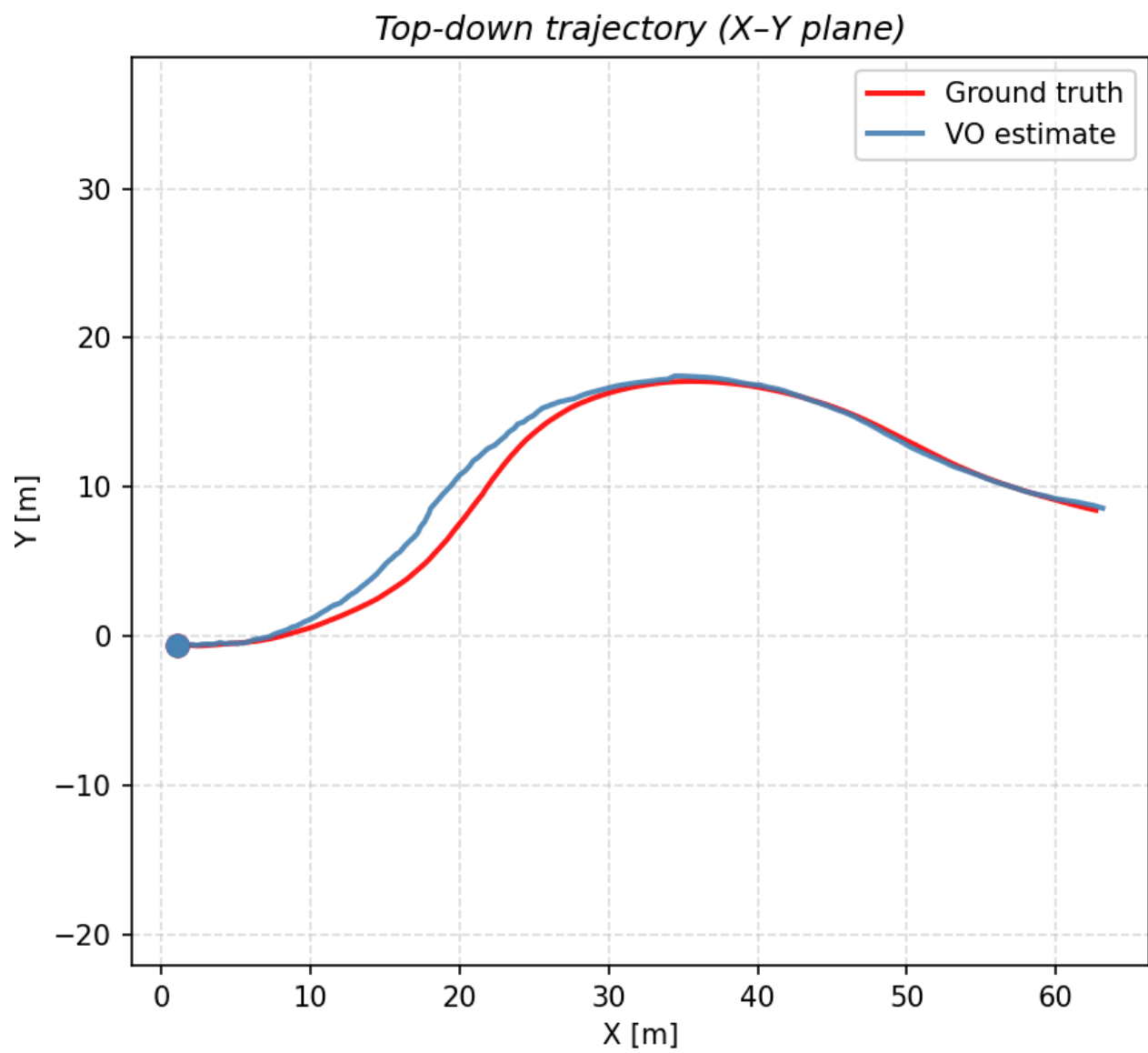


Figure 3: VO estimate of 2D trajectory vs ground truth

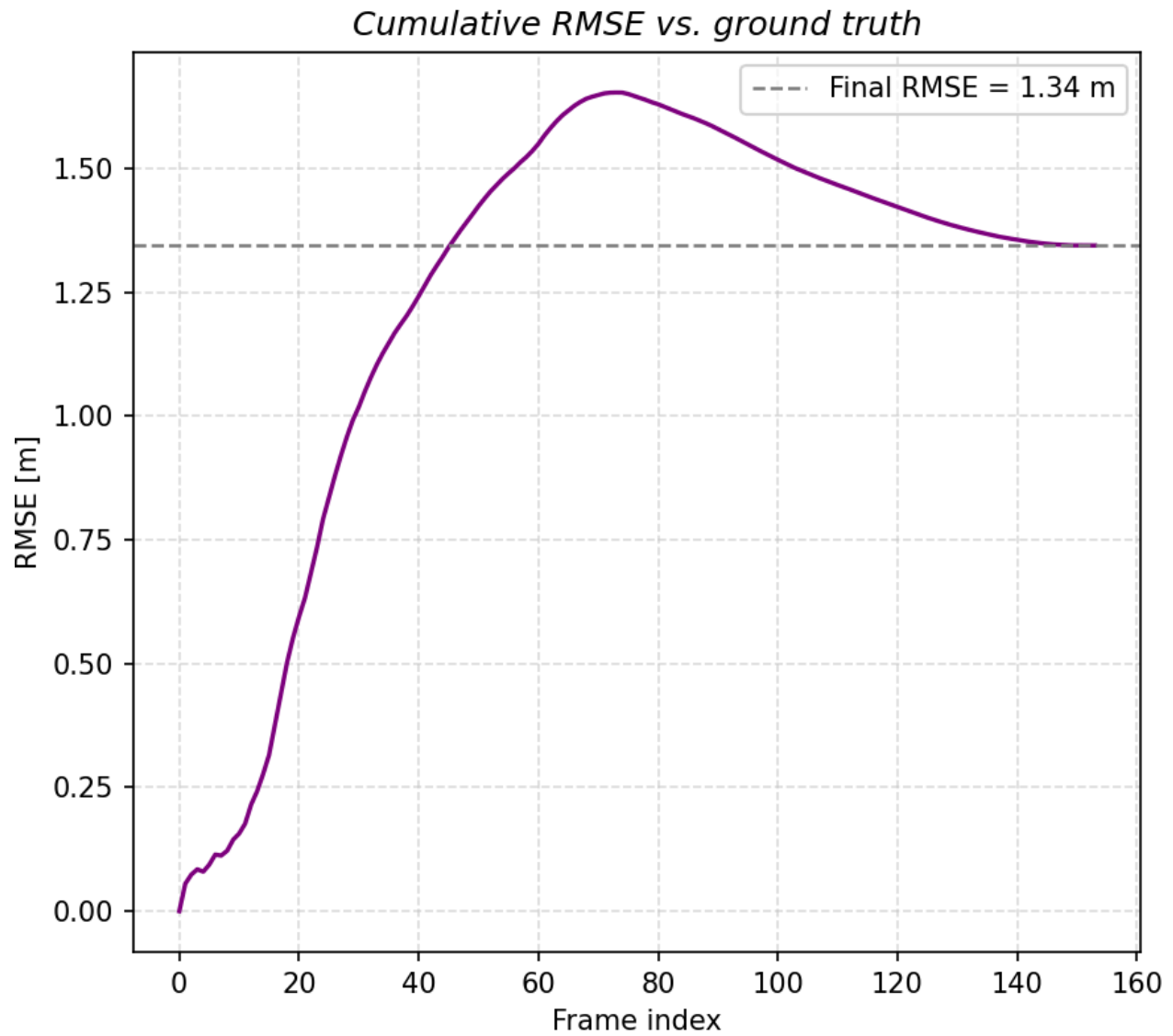


Figure 4: RMSE between VO estimate and Ground Truth

AI Acknowledgment

This project benefited from AI-assisted code organization and design critique. Claude was used to help clean up the structure of the codebase and provide feedback on architectural decisions, while all core ideas and implementation remain the work of the group members.

ChatGPT 5.2 was used for citation forming, and to aid understanding of concepts.